

Generative Imaging using Diffusion Models

Marios S. Pattichis, FEAMBES, Senior Member, National Academy of Inventors

Director of online M.Sc. In Computer Engineering in Applied Machine Learning And AI Systems Engineering

Department of Electrical and Computer Engineering

The University of New Mexico

References

The coding examples are adopted from:

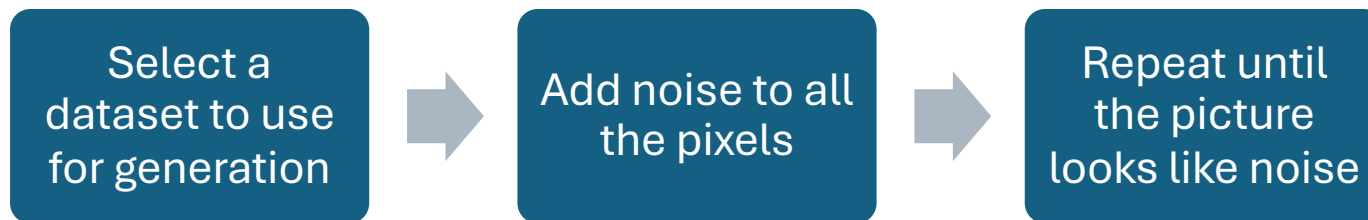
A. Géron (2025). *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly.

The basic **learning models** are from:

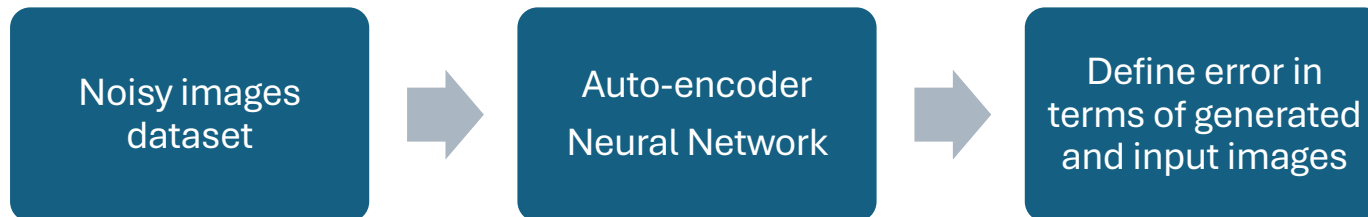
Torralba, A., Isola, P., & Freeman, W. T. (2024). *Foundations of Computer Vision*. The MIT Press. <https://visionbook.mit.edu/>.

The Diffusion Process: Training

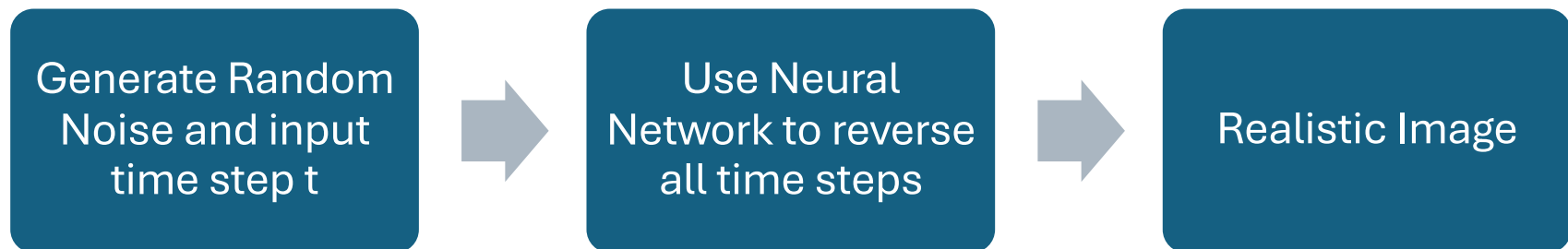
Step 1. Build a dataset to train the diffusion process:



Step 2. Train an auto-encoder neural network to remove noise from an image:



The Diffusion Process: Deployment



Training dataset: Create a sequence from any input image

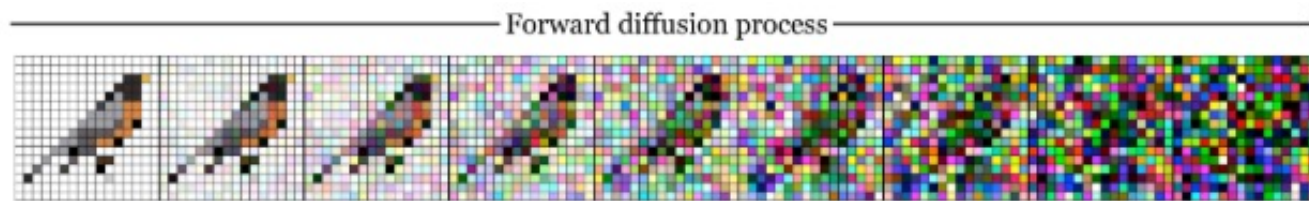


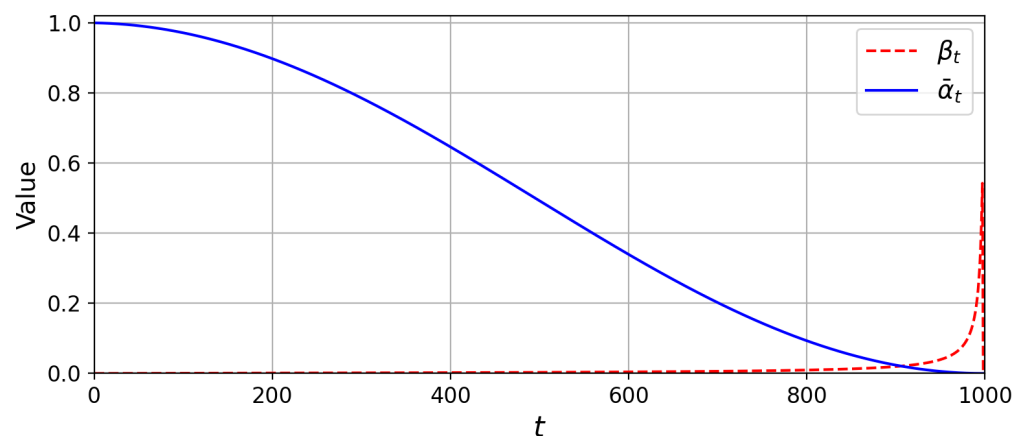
Figure 32.14: Forward diffusion process.

After T steps of this process, the image $\mathbf{x}_T$ looks like pure noise, if T is large enough. In fact, if we use the following noise process, then $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ as $T \rightarrow \infty$ (which follows from equation 4 in [4]).

$$\begin{aligned}\mathbf{x}_t &= \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t \\ \epsilon_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{I})\end{aligned}$$

Generate noisy videos

Code uses:
 $ab = 1 - \beta_t$
 $\beta_t = 1 - ab$



$$\mathbf{x}_t = \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t$$

$$\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Signal coefficient keeps going down.
 Noise coefficient picks up at the end.

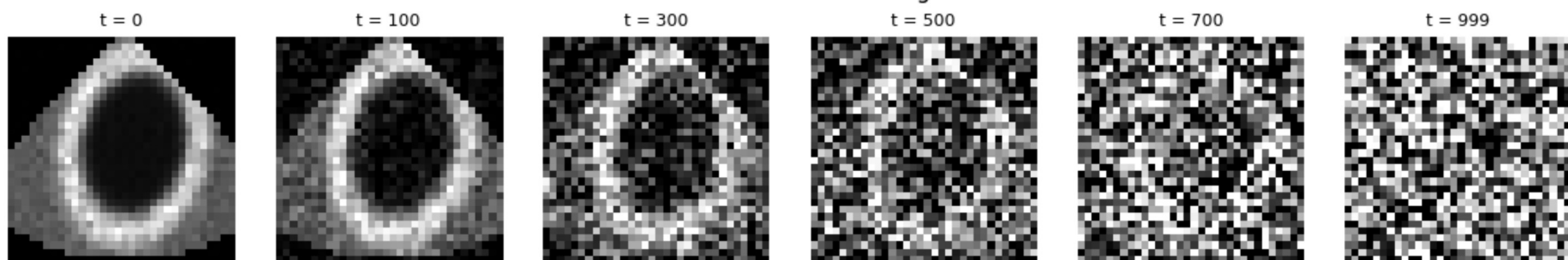
```
N_STEPS = 1000 # number of noise levels: t = 0, 1, ..., 999
s = torch.arange(N_STEPS + 1, device=device) / N_STEPS
alpha_bar = (torch.cos((s + 0.008) / 1.008 * math.pi / 2) ** 2) # cosine schedule
alpha_bar = (alpha_bar[1:] / alpha_bar[0]).clamp(1e-5, 1) # 1 = clean ... ~0 = pure noise

def add_noise(x0, t, noise): # x0: clean videos, shape (B, 1, T, H, W)
    ab = alpha_bar[t].view(-1, 1, 1, 1, 1) # one alpha_bar per video, shaped to match x0
    return ab.sqrt() * x0 + (1 - ab).sqrt() * noise # x_t = sqrt(ab)*x0 + sqrt(1-ab)*noise

# In the training loop:
noise = torch.randn_like(x0) # random Gaussian noise, same shape as the videos
xt = add_noise(x0, t, noise) # noisy videos at random noise levels t
```

Adding noise until it looks like noise

First frame of one video at increasing noise levels



Neural Net Goal: Reverse the sequence

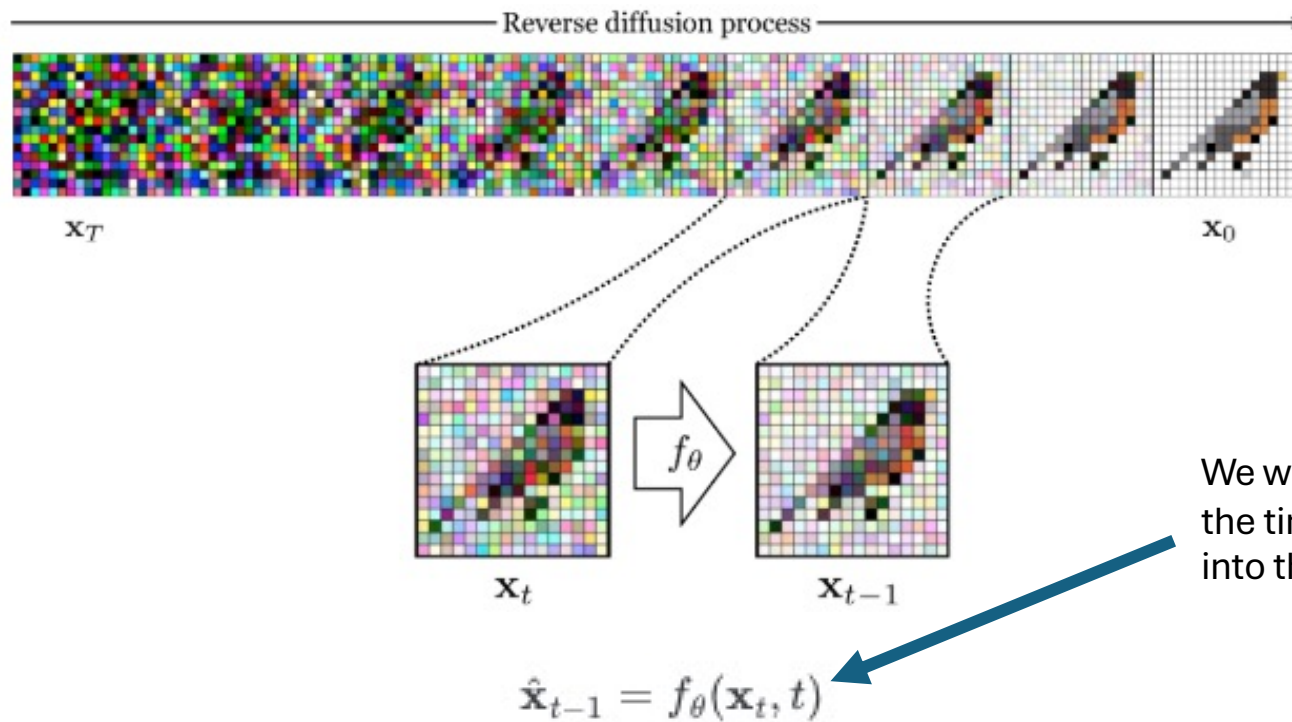


Figure 32.15: Reverse diffusion process.
The forward process creates supervision to train the reverse process.

We will look at embedding the time t and other parameters into the input.

<https://visionbook.mit.edu/>

Embedding Time using sinusoids

```
def timestep_embedding(t, dim):  
    half = dim // 2  
    freqs = torch.exp(-math.log(10000) * torch.arange(half, device=t.device) / half)  
    args = t.float()[ :, None] * freqs[None]  
    return torch.cat([args.sin(), args.cos()], dim=1)
```

Residual block: from c_in to c_out channels

The skip connection converts from input channels to output channels if needed, enabling a residual connection.

```
class ResBlock(nn.Module):
    def __init__(self, c_in, c_out, emb_dim):
        super().__init__()
        self.norm1, self.conv1 = nn.GroupNorm(8, c_in), nn.Conv3d(c_in, c_out, 3, padding=1)
        self.emb = nn.Linear(emb_dim, c_out)
        self.norm2, self.conv2 = nn.GroupNorm(8, c_out), nn.Conv3d(c_out, c_out, 3, padding=1)
        self.skip = nn.Conv3d(c_in, c_out, 1) if c_in != c_out else nn.Identity()
    def forward(self, x, e):
        h = self.conv1(F.silu(self.norm1(x)))
        h = h + self.emb(e)[:, :, None, None, None]
        h = self.conv2(F.silu(self.norm2(h)))
        return h + self.skip(x)
```

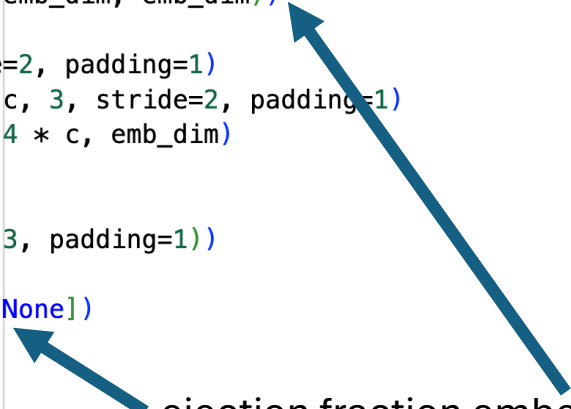
Residual connection.

ejection fraction embedding propagation through linear layer.

UNet3D: Autoencoder with time and ef embeddings

```
class UNet3D(nn.Module):
    def __init__(self, c=16, emb_dim=128):
        super().__init__()
        self.emb_dim = emb_dim
        self.t_mlp = nn.Sequential(nn.Linear(emb_dim, emb_dim), nn.SiLU(), nn.Linear(emb_dim, emb_dim))
        self.ef_mlp = nn.Sequential(nn.Linear(1, emb_dim), nn.SiLU(), nn.Linear(emb_dim, emb_dim))
        self.inc = nn.Conv3d(1, c, 3, padding=1)
        self.d1, self.down1 = ResBlock(c, c, emb_dim), nn.Conv3d(c, c, 3, stride=2, padding=1)
        self.d2, self.down2 = ResBlock(c, 2 * c, emb_dim), nn.Conv3d(2 * c, 2 * c, 3, stride=2, padding=1)
        self.mid1, self.mid2 = ResBlock(2 * c, 4 * c, emb_dim), ResBlock(4 * c, 4 * c, emb_dim)
        self.u2 = ResBlock(4 * c + 2 * c, 2 * c, emb_dim)
        self.u1 = ResBlock(2 * c + c, c, emb_dim)
        self.out = nn.Sequential(nn.GroupNorm(8, c), nn.SiLU(), nn.Conv3d(c, 1, 3, padding=1))

    def forward(self, x, t, ef):
        e = self.t_mlp(timestep_embedding(t, self.emb_dim)) + self.ef_mlp(ef[:, None])
        up = lambda h: F.interpolate(h, scale_factor=2, mode="nearest")
        h1 = self.d1(self.inc(x), e) # 16 x 32 x 32
        h2 = self.d2(self.down1(h1), e) # 8 x 16 x 16
        h = self.mid2(self.mid1(self.down2(h2), e), e) # 4 x 8 x 8
        h = self.u2(torch.cat([up(h), h2], 1), e)
        h = self.u1(torch.cat([up(h), h1], 1), e)
        return self.out(h) # predicted v, same shape as x
```



ejection fraction embedding.

Training EMA using UNet3D

```
STEPS, BATCH, LR, EMA_DECAY = 1500, 32, 5e-4, 0.995
```

```
use_amp = device == "cuda"
```

```
opt = torch.optim.AdamW(model.parameters(), lr=LR)
```

```
ema = copy.deepcopy(model).eval().requires_grad_(False)
```

```
scaler = torch.amp.GradScaler(enabled=use_amp)
```

ema model.

```
losses, t0 = [], time.time()
```

```
model.train()
```

```
for step in range(1, STEPS + 1):
```

Add noise and train UNet3D to reverse.

```
    x0, ef = make_echo(BATCH)
```

```
    t = torch.randint(0, N_STEPS, (BATCH,), device=device)
```

```
    noise = torch.randn_like(x0)
```

```
    ab = alpha_bar[t].view(-1, 1, 1, 1)
```

```
    v_target = ab.sqrt() * noise - (1 - ab).sqrt() * x0
```

```
    with torch.autocast(device_type=device, dtype=torch.float16, enabled=use_amp):
```

```
        loss = F.mse_loss(model(add_noise(x0, t, noise), t, ef).float(), v_target)
```

```
    opt.zero_grad(set_to_none=True)
```

```
    scaler.scale(loss).backward()
```

```
    scaler.step(opt); scaler.update()
```

```
    with torch.no_grad():
```

```
        for p_ema, p in zip(ema.parameters(), model.parameters()):
```

```
            # EMA update
```

```
            p_ema.lerp(p, 1 - EMA_DECAY)
```

Train ema model.

EMA is an **exponential moving average of the model's weights**. In the diffusion notebook, a second copy of the U-Net slowly follows the one being trained, and that smoothed copy is the one used to **generate** the videos.

The code

```
ema = copy.deepcopy(model).eval().requires_grad_(False)    # a frozen copy of the U-Net
...
# after every training step:
for p_ema, p in zip(ema.parameters(), model.parameters()):
    p_ema.lerp_(p, 1 - EMA_DECAY)                          # EMA_DECAY = 0.995
```

`lerp_` means "linear interpolation, in place". For every weight:

$$w_{\text{EMA}} \leftarrow 0.995 \cdot w_{\text{EMA}} + 0.005 \cdot w_{\text{model}}$$

After each step, the EMA copy moves **0.5% of the way** toward the current weights. Older values fade gradually (exponentially), so the EMA copy is effectively an average of the weights over roughly the **last $1 / (1 - 0.995) = 200$ steps**.

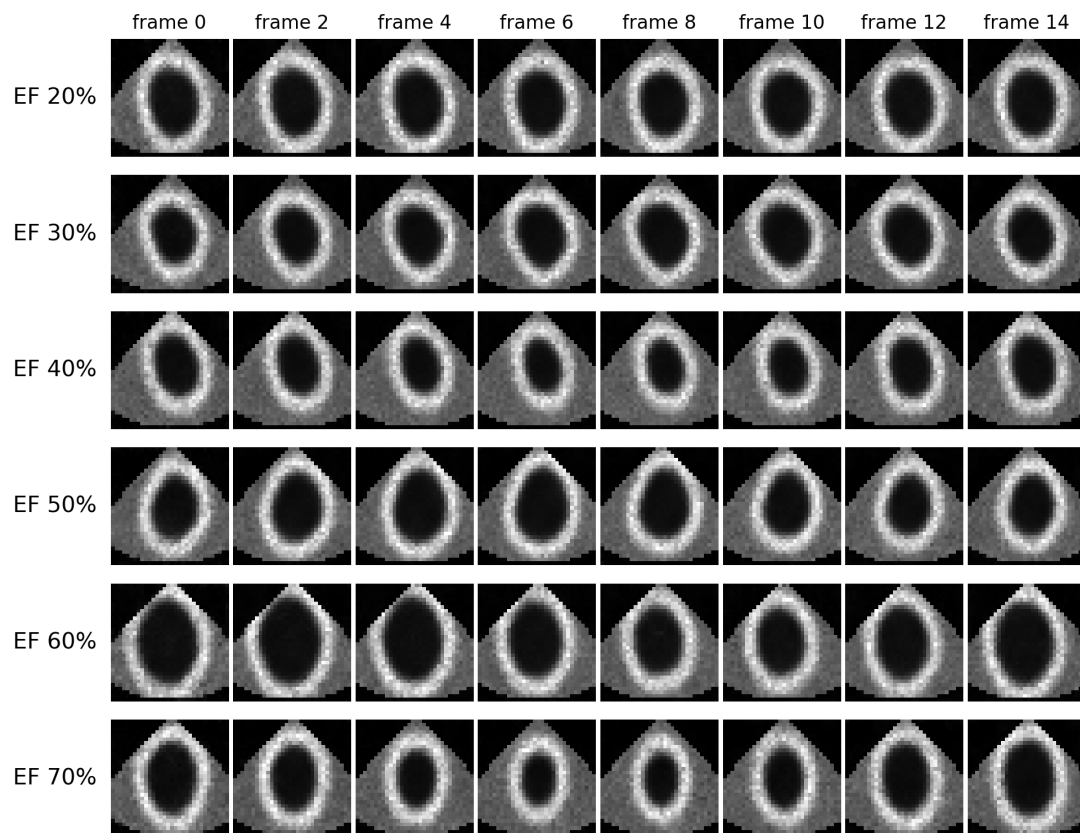
The EMA model is never trained directly: it gets no gradients (`requires_grad_(False)`), and the optimizer doesn't touch it. It only follows.

Use ema to generate the target with given ef

```
@torch.no_grad()
def generate(ef, steps=50, net=None):
    net = net or ema                                # use the EMA weights
    ef = ef.to(device)
    x = torch.randn(len(ef), 1, T, S, S, device=device) # start from pure noise
    ts = torch.linspace(N_STEPS - 1, 0, steps, device=device).long()
    for i, t in enumerate(ts):
        ab = alpha_bar[t]
        ab_next = alpha_bar[ts[i + 1]] if i + 1 < steps else torch.tensor(1.0, device=device)
        with torch.autocast(device_type=device, dtype=torch.float16, enabled=use_amp):
            v = net(x, t.repeat(len(x)), ef).float()
            x0 = (ab.sqrt() * x - (1 - ab).sqrt() * v).clamp(-1, 1) # estimate of the clean video
            eps = (x - ab.sqrt() * x0) / (1 - ab).sqrt()
            x = ab_next.sqrt() * x0 + (1 - ab_next).sqrt() * eps # DDIM step to the next level
    return x

t0 = time.time()
requested = torch.tensor([0.2, 0.3, 0.4, 0.5, 0.6, 0.7]) ← Generate these ejection fractions.
fake = generate(requested)
print(f"Generated {len(fake)} videos in {time.time() - t0:.1f} s")
animate(fake, [f"generated, EF {e:.0%}" for e in requested])
```

Example Output: Heart Videos

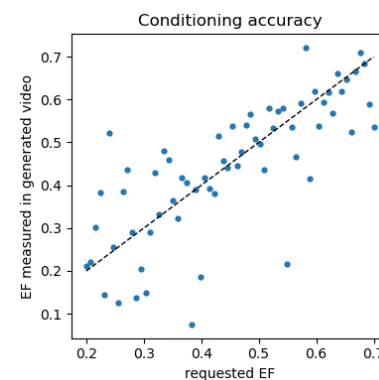


Ejection fraction (EF): the fraction of blood the left ventricle pumps out with each heartbeat.

EF = (EDV - ESV) / EDV EDV, ESV: ventricle volume when most filled / most emptied

Normal: about 55–70%. 40% or less: reduced (heart failure).

In our videos (2D): EF = 1 - smallest cavity area / largest cavity area



Above: requested EF vs. EF measured in 64 generated videos.

Correlation 0.77, mean error 0.07

Do the videos appear realistic? Does each heart pump as requested?